

2026-Fall-DL-hw2

October 5, 2026

1 Homework-2

- CSCI-4931 : Deep Learning
- Fall 2026
- Instructor: Ashis Kumer Biswas, Ph.D.

```
[1]: # %pip install -f requirements.txt
!which python
```

/home/mark/anaconda3/envs/deep_learning/bin/python

- Please assign your name, and declare collaborators (if any) to the variables below:

```
[1]: my_name = "Mark Evers"
```

1.1 Little Background about the problem

‘Customer churn’ occurs when customers stop doing business with a company, also known as customer attrition. It is also referred to as loss of clients or customers.

You are given sensitive information of 9,000 of an European Bank, EBQ. Your task is to build an Artificial Neural Network (ANN) based on the dataset such that later the ANN model can predict correctly who is going to leave next. This predictive analysis is vital for the EBQ bank to revise their business strategy towards customer retention. What do you think?

Anyway, you are recruited by the bank to do the data science. And, the head of the bank only trusts heads, i.e., brains.... I mean neural networks for making any decisions. And luckily you were in Dr. B’s class and you know something(?) about the ANN that you could successfully convince the head of the bank during the interview. He has put a lot of faith in you. Now, can you solve his problem?

Tasks:

1. Please download the zip file, ‘hw2-deliverables.zip’. Unzip it in your workspace. Here below is the file hierarchy of “hw2-deliverables/” folder:

```
hw2-deliverables
  2026-Fall-DL-hw2.ipynb
  dataset
    datasetX.csv
  figures
    le.png
```

```
nn-1.png
nn-1.svg
nn-2.png
nn-2.svg
nn-3.png
nn-3.svg
ohe.png
saved_models
```

As you can see you will mostly be working with the `2026-Fall-DL-hw2.ipynb`, i.e., the jupyter notebook. The notebook accesses the dataset files: `dataset/datasetX.csv` containing few customer information and is labeled (i.e., the target column, `Exited` is present). Here below is a brief summary of the features you will find in the datasets:

- **CustomerId**: a unique identifier for each customer within the dataset. These values are not ordered sequentially within the dataset, and are only used to identify a specific customer. It typically does not have any influence to whether a customer leaves the business.
- **Surname**: A string used to identify the customer in the dataset. Surname may be distinct amidst all or most customers. Because of this, it most likely won't affect the target variable.
- **CreditScore**: a numeric representation of the customer's individual fiscal credit score. Typically used to indicate eligibility for loans. Current credit scores use a range from 300 to 850, but the FICO auto score range uses 250-900. This feature likely determines retention rate of customers.
- **Geography**: this feature contains a categorical string representing the name of a country the customer is from originally.
- **Gender**: this feature contains a categorical string representing the gender of the customer ("Male"/"Female").
- **Age**: a numerical integer representation of a customer's age. Intuition suggests that older customers are likely to have higher retention than younger customers.
- **Tenure**: a numerical integer representation. It is assumed that this feature represents the number of total years the customer has been retained. It is likely that customers which have been retained longer will continue to be retained.
- **Balance**: a numerical floating point number (to two decimal places of precision) indicating the customer's current bank balance (assumed total across all accounts). Customers with a greater balance may be less likely to exit the account due to difficulty of transfer.
- **NumOfProducts**: numeric integer value. It is assumed that this value represents the number of accounts (products) that this customer has open. Further evaluation of this feature would be needed to determine the usefulness of this feature, but at face-value, intuition dictates that a customer with more products is less likely to exit.
- **HasCrCard**: boolean flag (0 or 1) representing whether the customer has a credit card or not.
- **IsActiveMember**: boolean flag (0 or 1) representing whether the customer is an active member of the bank. It is assumed this indicates whether the customer has transactions on the regular banking statement. Intuition dictates that inactive members are more likely to exit.
- **EstimatedSalary**: numerical floating point representation of the customer's predicted salary (to two decimal places) intuition dictates that customers with different incomes may behave differently with respect to retention rate.
- **Exited**: boolean flag (0 or 1) representing whether the customer has exited their account. This is the target variable for the dataset. It should not be dropped, but should not be included as the training input (X), and should instead be separated as the target label (y).

You will also see an empty directory `saved_models/`, that is for you to save all the models you'd train in this assignment.

`figures/` directory contains few image files used to properly document this assignment. Please do not delete and when possible please move them with this jupyter notebook for proper display of its contents.

In this Jupyter notebook please write your solutions / codes in the cells marked with **#Your solution goes here....** You may add additional code cells after that cell if you desire. But, please do not remove any cell originally given in the notebook.

After you solve the assignment in the jupyter notebook, be sure to execute and save it so that execution/results/printouts are also saved with it. Finally, submit the saved jupyter notebook (2026-Fall-DL-hw2.ipynb) in Canvas to receive grade. Optionally, you can also submit a python version of the notebook if desired. For this assignment, Canvas only will accept either jupyter notebook in `*.ipynb` or python script (`*.py`) extensions.

1.2 Task 1 : (10 points)

- Define a function named `summarize_dataset` that takes only one argument: `csv_file`, where `csv_file` is the name of the given csv file with this assignment, i.e., `datasetX.csv`.
 - The function is expected to summarize the given dataset in the following way:
total number of rows = a
total number of columns = b
number of columns having non-numeric values = c
columns with missing values = [(d1, e1) (d2, e2), ...]
gender based summary of exited column = [(f1, g1) (f2, g2), ...]
age based summary of exited column = [('below or equal to 40', h1) ('above 40', h2)]
credit score summary = i +/- j

where,

- a is total number of rows in the dataset.
- b is total number of columns in the dataset.
- c is number of columns having non-numeric values.
- (d_i, e_i) (i.e., a pair/tuple entry) represents column name (d_i) and number of missing values present in that column (e_i). If number of missing values in a column is zero (0), you do not need to list it. Please sort the tuple entries in descending order of e_i values.
- g_i represents the percentage of gender f_i who exited. Please sort the tuple entries in descending order of g_i values. Also, print the percentages in 2 decimal places after the decimal point, and print use % symbol after the percentage value.
- h_1 and h_2 represents the percentage of ≤ 40 year olds who exited, and the percentage > 40 year olds who exited. Also, print the percentages in 2 decimal places after the decimal point, and print use % symbol after the percentage value.
- j and k are average and standard deviation of credit scores among the data samples respectively. Please print the way it is shown above. Also, print the both values in 2 decimal places after the decimal point.

```

[2]: import pandas as pd

def summarize_dataset(csv_file):

    # load the dataset
    df = pd.read_csv(csv_file)
    df_len = len(df)

    # print the number of rows and columns
    print(f"Number of rows: {df.shape[0]}")
    print(f"Number of columns: {df.shape[1]}")

    # print the number of columns having non-numeric values
    non_numeric_columns = df.select_dtypes(exclude=['number']).columns
    print(f"Number of columns with non-numeric values: {len(non_numeric_columns)}")

    # print the columns with missing values
    missing_values_cols = df.columns[df.isnull().any()]
    missing_values_count = df[missing_values_cols].isnull().sum()
    print("Columns with missing values:")
    for col, count in missing_values_count.items():
        print(f"    {col}: {count} missing values")

    # print the percentage of each gender that exited
    print("Percentage of each gender that exited:")
    for gender in df['Gender'].unique():
        gender_df = df[df['Gender'] == gender]
        exited_percentage = (gender_df['Exited'].sum() / df_len) * 100
        print(f"    {gender}: {exited_percentage:.2f}%")

    # print the percentage of under/over 40 years old that exited
    under_40_exited = df[df['Age'] <= 40]['Exited'].sum() / df_len * 100
    print(f"Percentage of under/over 40 years old that exited: {under_40_exited:.2f}%")

    over_40_exited = df[df['Age'] > 40]['Exited'].sum() / df_len * 100
    print(f"Percentage of over 40 years old that exited: {over_40_exited:.2f}%")

    # print the credit score mean and standard deviation
    print(f"Credit score mean and stddev: {df['CreditScore'].mean():.2f} +/- {df['CreditScore'].std():.2f}")

    # return the dataframe
    return df

# get the dataset summary and save the dataframe to a variable

```

```
df = summarize_dataset("dataset/datasetX.csv")

# fix the missing values
credit_score_mean = df['CreditScore'].mean()
age_median = df['Age'].median()
df.fillna({'CreditScore': credit_score_mean, 'Age': age_median}, inplace=True)

pass # this is so it doesn't print the whole table...
```

```
Number of rows: 9000
Number of columns: 13
Number of columns with non-numeric values: 3
Columns with missing values:
    CreditScore: 26 missing values
    Age: 397 missing values
Percentage of each gender that exited:
    Male: 9.04%
    Female: 11.30%
Percentage of under/over 40 years old that exited: 6.54%
Percentage of over 40 years old that exited: 13.47%
Credit score mean and stddev: 650.25 +/- 96.75
```

1.3 Task 2

- Preprocessing the given dataset for the model training.

1.3.1 Task 2.1 (10 points)

- First preprocessing that we are going to do on the dataset is dropping two features (i.e., columns) that, I think, are irrelevant and would not make any meaningful relationship with the `Exited` feature. The features are: `CustomerId` and `Surname`.
- Make sure to create a variable called `dataset_dropped` that will store the revised dataset.
- Please print the name of the columns of the revised dataset.

```
[3]: dataset_dropped = df.drop(columns=['CustomerId', 'Surname'])
print(f"Remaining columns: {dataset_dropped.columns.tolist()}")
```

```
Remaining columns: ['CreditScore', 'Geography', 'Gender', 'Age', 'Tenure',
'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary',
'Exited']
```

1.3.2 Task 2.2 (10 points)

- Second Preprocessing that we are going to do is *Shuffle Rows* of the dataset obtained from Task 2.1.
- “It is extremely important to shuffle the training data, so that you do not obtain entire minibatches of highly correlated examples. As long as the data has been shuffled, everything should work OK. Different random orderings will perform slightly differently from each other but this will be a small factor that does not matter much.” – [Ian Goodfellow](#)

- Use a random seed value 4321 in case you will call any stochastic method.
- Make sure to create a variable called `dataset_shuffled` that will store the revised dataset.

```
[4]: import numpy as np

np.random.seed(4321)
dataset_shuffled = dataset_dropped.sample(frac=1).reset_index(drop=True)
```

1.3.3 Task 2.3: (10 points)

- Third Preprocessing that we will do is X-y Partitioning of the dataset obtained from Task 2.2.
- In its current state, the dataset contains both independent (input, X) and the target (output, y) features within the same dataframe. For ease of the training process, we need to partition the training features from the target feature into two separate dataframes.
- Make sure, the following cell contains at least two variables: X and y:
 - X contains part of the dataset with only independent features, and
 - y having only the dependent/target feature.

```
[5]: X = dataset_shuffled.drop(columns=['Exited'])
y = dataset_shuffled['Exited']
```

1.3.4 Task 2.4 (10 points)

- Fourth Preprocessing that we will do is Train-Test Split of X, y obtained from Task 2.3.
- Now that we have X and y tables with appropriate feature pruning performed, we must split the data into a training partition (X_train, y_train) and a testing partition (X_test, y_test).
- The training partitions (X_train, y_train) will be used to train your model, while the test partition (X_test, y_test) will be set aside during the training steps, and will only be used to evaluate the trained model.
- Training and test splits should be mutually exclusive to the datasets... i.e., a sample can not be both in training and test sets.
- Please perform a 80-20 split, meaning 80% of the (X,y) dataset will be in (X_train, y_train) split, while, remaining 20% will be in (X_test, y_test) split.
- Please use random seed 4321 prior to calling any stochastic methods.
- Make sure the following cell contains at least 4 variables: X_train, y_train, X_test, y_test.

```
[6]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

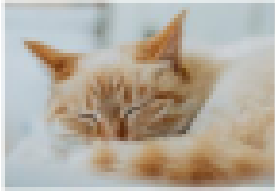
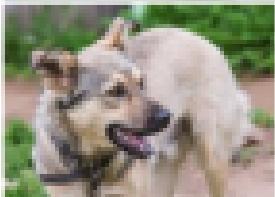
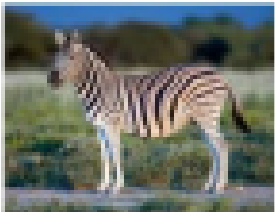
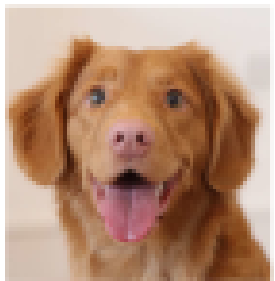
1.3.5 Task 2.5 (10 points)

- Fifth preprocessing that we will do is the *Conversion of Categorical features to Numerical*
- Please adopt the One Hot Encoding method instead of Label Encoding while converting the categorical features.

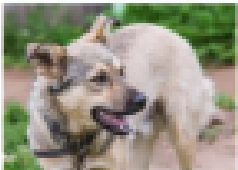
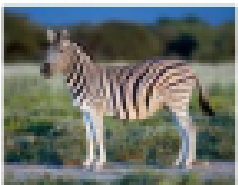
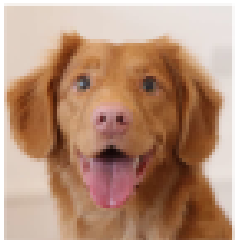
- Make sure the following cell contains a variable named `X_train_ohe` that would contain one hot encoded `X_train` data; on the two categorical columns: ‘Geography’, ‘Gender’. Please save the encoder for later use; e.g., encode `X_test` dataset, or any future test sample given to you. Under any circumstance, you must not encode `X_test` independently like you would do for `X_train`.
- Now, encode the `X_test` data using the one hot encoder you saved while you encoded the `X_train`, and name the variable `X_test_ohe`.
- **Both encoding techniques are outlined below:** > A little background first: Categorical features are features that contain values that are not numeric. It would be absurd to work with non-numeric features if you ask neurons in your ANN to compute the weighted sum of inputs, and then pass through activation function, right? These maths are undefined. An obvious solution you may be intrigued to do is dropping the features! Aha! Wrong!! Every piece of data is precious... may present with valuable insights of the data samples to find the patterns to map inputs with output/targets. So, we should include them. But, how?

The answer is via “Encoding”.

Several types of encodings are used in practice. Here below are just 2 popular ones: 1. **Label Encoding**, where labels are encoded as subsequent numbers. Say, for a categorical feature named “Category” with three categorical values: {“Cat”, “Dog” or “Zebra”} can be encoded to “0”, “1”, “2” respectively as in figure below. The issue with this type of encoding may unintentionally impose a type of ordering of the categories, that may add bias to the training.

Sample	Category	Encoding
	Cat	0
	Dog	1
	Zebra	2
	Dog	1

2. **One Hot Encoding**, ignores the ordering of the categories all together. With one-hot, we convert each categorical value into a new categorical column and assign a binary value of 1 or 0 to those columns. Each integer value is represented as a binary vector. All the values are zero, and the index is marked with a 1. Also, don't forget to remove the original categorical features. Here below just an example, how to convert the categorical feature called "Category" having the {"Cat", "Dog" or "Zebra"} values into three new binary features: "Cat", "Dog", "Zebra".

Sample	Category	One-Hot Encoding		
		Cat	Dog	Zebra
	Cat	1	0	0
	Dog	0	1	0
	Zebra	0	0	1
	Dog	0	1	0

A note on the Dummy Variable Trap The Dummy Variable Trap occurs when two or more dummy variables created by one-hot encoding are highly correlated (i.e., becomes multi-collinear). This means that one variable can be predicted from the others, making it difficult to interpret predicted coefficient variables in regression models. In other words, the individual effect of the dummy variables on the prediction model can not be interpreted well because of multicollinearity.

Using the one-hot encoding method, a new dummy variable is created for each categorical variable to represent the presence (1) or absence (0) of the categorical variable. For example, if tree species is a categorical variable made up of the values pine, or oak, then tree species can be represented as a dummy variable by converting each variable to a one-hot vector. This means that a separate column is obtained for each category, where the first column represents if the tree is pine and the second column represents if the tree is oak. Each column will contain a 0 or 1 if the tree in question is of the column's species. These two columns are multi-collinear since if a tree is pine, then we know it's not oak and vice versa. The machine learning models trained on dataset having this multi-collinearity suffers. A remedy is to drop first (or any one) of the dummy (i.e., one-hot) features created.

```
[7]: from sklearn.preprocessing import OneHotEncoder, MinMaxScaler, StandardScaler
from sklearn.compose import ColumnTransformer

# create a preprocessor to encode the columns we need
preprocessor = ColumnTransformer(
    transformers=[
        ("onehot", OneHotEncoder(sparse_output=False, drop='first',
    ↪handle_unknown='ignore'), ["Geography", "Gender"]),
        ("zscaler", StandardScaler(), ["CreditScore", "Balance",
    ↪"EstimatedSalary"]),
        ("minmax", MinMaxScaler(), ["Age", "Tenure", "NumOfProducts"])
    ],
    remainder="passthrough"
)

# # create a preprocessor to encode the columns we need (minmax only)
# preprocessor = ColumnTransformer(
#     transformers=[
#         ("onehot", OneHotEncoder(sparse_output=False, drop='first',
    ↪handle_unknown='ignore'), ["Geography", "Gender"]),
#         ("minmax", MinMaxScaler(), ["CreditScore", "Balance",
    ↪"EstimatedSalary", "Age", "Tenure", "NumOfProducts"])
#     ],
#     remainder="passthrough"
# )

# create a preprocessor to encode the columns we need (zscaler only)
preprocessor = ColumnTransformer(
    transformers=[
        ("onehot", OneHotEncoder(sparse_output=False, drop='first',
    ↪handle_unknown='ignore'), ["Geography", "Gender"]),
        ("zscaler", StandardScaler(), ["CreditScore", "Balance",
    ↪"EstimatedSalary", "Age", "Tenure", "NumOfProducts"]),
    ],
    remainder="passthrough"
)

# fit and transform the training and then transform the test data
X_train_ohe = preprocessor.fit_transform(X_train)
X_test_ohe = preprocessor.transform(X_test)

print(f"Column names before: {X.columns.tolist()}")
print(f"Column names after: {preprocessor.get_feature_names_out()}")
```

Column names before: ['CreditScore', 'Geography', 'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember', 'EstimatedSalary']

Column names after: ['onehot__Geography_Germany' 'onehot__Geography_Spain' 'onehot__Gender_Male' 'zscaler__CreditScore' 'zscaler__Balance'

```
'zscaler__EstimatedSalary' 'zscaler__Age' 'zscaler__Tenure'
'zscaler__NumOfProducts' 'remainder__HasCrCard'
'remainder__IsActiveMember']
```

1.3.6 Task 2.6: (10 points)

- Sixth Preprocessing that we are going to do is *Normalization of X_{train_ohe} , and X_{test_ohe}*
- Now that we have all numerical training and test datasets: X_{train_ohe} and X_{test_ohe} respectively, we can normalize each features in both of the datasets. **Normalization** is just one of the way to scale each feature. In class you'll learn a ton of other ways to scale. For this task, let's resort to **Normalization**.

“The rule of thumb for scaling datasets, is we scale training dataset first, then using the statistics that we learn during the scaling process, we scale the test dataset. We do not learn any new statistics while we scale the test dataset.”

- Also, scaling is commonly performed column-wise, and never sample/row wise.
- Make sure the following cell contains the two scaled variables: X_{train_scaled} and X_{test_scaled} based on the requirements mentioned above.

```
[8]: print("I used a `ColumnTransformer` to preprocess the data. The scalers were_
bundled with the `OneHotEncoder` in the cell above!")

X_train_scaled = X_train_ohe
X_test_scaled = X_test_ohe
```

I used a `ColumnTransformer` to preprocess the data. The scalers were bundled with the `OneHotEncoder` in the cell above!

1.4 Task 3: (10 points)

- *Designing your first Artificial Neural Network (ANN) based classifier using PyTorch.:*

1.4.1 Step 0: Dataset and DataLoaders

- Make sure you implement the Dataset and DataLoader classes ready to read from the external csv file located at dataset/datasetX.csv file.

```
[ ]: import torch
from torch.utils.data import Dataset, DataLoader

class BankChurnDataset(Dataset):
    def __init__(self, X, y):
        # Convert raw data (e.g., NumPy arrays) into PyTorch Tensors
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)
```

```

def __len__(self):
    # Return the total number of data samples
    return len(self.X)

def __getitem__(self, i):
    # Fetch the sample and label at the specified index
    return self.X[i], self.y[i]

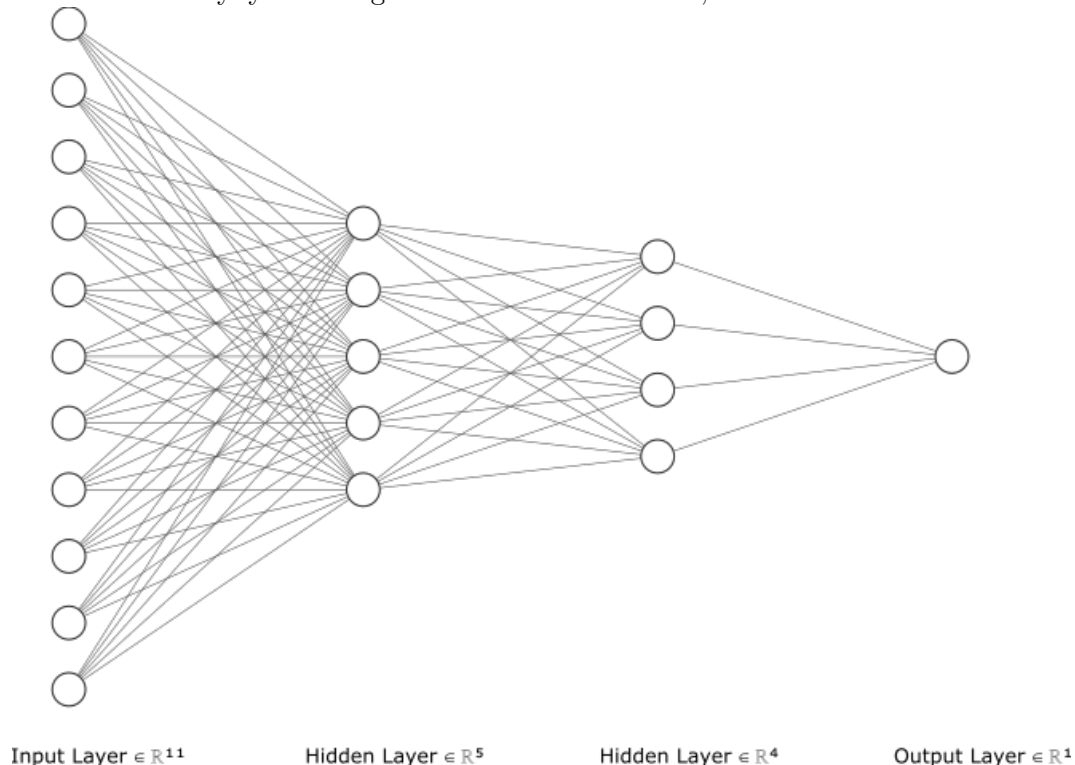
# populate the datasets using the global variables
train_dataset = BankChurnDataset(X_train_scaled, y_train.values)
test_dataset = BankChurnDataset(X_test_scaled, y_test.values)

# wrap them in DataLoaders
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

```

1.4.2 Step 1: The ANN architecture

- Let's design the first artificial network architecture for the classifier we would like to build. Here below is one. How did I get this architecture? Maybe in my dream! Haha. Someday you will get one too. Until that, let's follow the architecture below:



- **Input layer** will have 11 units as the dimension of training set: `X_train_scaled` (i.e, number of columns = 11).
- **First hidden layer** will have 5 neurons, each with “Rectified Linear Unit (‘ReLU’)” as activation function.

- **Second hidden layer** will have 4 neurons, each with “ReLU” as activation function.
- **Output layer** will have just 1 neuron, with ‘sigmoid’ activation function.
 - * The reason behind a single neuron with **sigmoid** activation at the output layer is that, output of this neuron will tell the probability score of the target outcome: “Exited” True or False. If the output neuron produces value above 0.5, we will say the neural network predicted “True”, otherwise, False. This is the beauty of using sigmoid function at the output layer as we can interpret the output value of the neuron as probability score.
- The architecture will come to life when you initiate the training process with training data.
 - The training process needs a **gradient descend based optimizer**, and a convex looking **loss function**.
 - For this task, let’s choose the **adam** optimizer, and the **binary_crossentropy** as the loss function.
 - You choose a batch size that would make the execution comfortable at your workstation.

```
[10]: import torch.nn as nn
import torch.optim as optim

class BankChurnANN(nn.Module):

    def __init__(self):

        super().__init__()

        # define the layers
        self.hidden1 = nn.Linear(11, 5)
        self.hidden2 = nn.Linear(5, 4)
        self.output = nn.Linear(4, 1)

        # define the activation functions
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # pass the data through layers and activations
        x = self.relu(self.hidden1(x))
        x = self.relu(self.hidden2(x))

        # sigmoid to return a probability between 0 and 1
        x = self.sigmoid(self.output(x))

        # return the output
        return x.squeeze(-1) # remove any extra dimensions for compatibility
↪with BCELoss
```

```

# make an instance of the model
model = BankChurnANN()

# define the loss function and optimizer
loss_function = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

1.4.3 Step 2: The Training process

- Let's start the training process with the training dataset, `X_train_scaled`.
 - Gradient descend based optimization updates run in iterations. When number of iterations equal the total number of training samples, we call that 1 **epoch** has passed. Let's continue the training for 25 **epochs**. But, you are welcome to run longer than this. There are, however, simpler way to determine if you should early stop your training.
 - * (Optional) Can you extract information about optimization in each epoch? If so, draw a epoch-loss plot, where X-axis needs to show epoch numbers, and Y-axis will show the **binary_crossentropy** loss value in that particular epoch iteration.
- Don't forget to save the model into a file in the `saved_models/` directory so that you can re-use it later for further prediction. Let's give it a name: `model-ann-11-5-4-1-pt` with an extension of your choosing.

```

[ ]: # Ensure labels in the dataset/dataloader are float32 for BCELoss
# (BCELoss expects predictions and targets to share the same float type)

# how long to train the model for
n_epochs = 25

def train_model(model, train_loader, loss_function, optimizer, n_epochs,
    ↪model_save_path):

    model
    # set the model to training mode
    model.train()
    # track the loss for each epoch
    epoch_losses = []

    # main training loop
    for epoch in range(n_epochs):
        # the running total loss for the epoch
        epoch_total_loss = 0.0

        # batch training loop
        for batch_x, batch_y in train_loader:

```

```

# Match target tensor dtype to model output dtype (float32)
# batch_y = batch_y.float()

# reset gradients from the previous batch
optimizer.zero_grad()
# run the forward pass through the network
predictions = model(batch_x)
# calculate the loss
loss = loss_function(predictions, batch_y)
# backpropagation
loss.backward()
# update weights and biases
optimizer.step()

epoch_total_loss += loss.item() * batch_x.size(0)

epoch_loss = epoch_total_loss / len(train_loader.dataset)
epoch_losses.append(epoch_loss)
print(f"Epoch [{epoch+1}/{n_epochs}] - Loss: {epoch_loss:.4f}")

# Save the trained model to the required directory
model.eval() # Set the model to evaluation mode before saving
torch.save(model, model_save_path)

```

```

[12]: %%time

# call the function to train the model and save it
train_model(model, train_loader, loss_function, optimizer, n_epochs,
↳ "saved_models/model-ann-11-5-4-1-pt.ann")

```

```

Epoch [1/25] - Loss: 0.5413
Epoch [2/25] - Loss: 0.4197
Epoch [3/25] - Loss: 0.3905
Epoch [4/25] - Loss: 0.3795
Epoch [5/25] - Loss: 0.3735
Epoch [6/25] - Loss: 0.3702
Epoch [7/25] - Loss: 0.3670
Epoch [8/25] - Loss: 0.3650
Epoch [9/25] - Loss: 0.3634
Epoch [10/25] - Loss: 0.3614
Epoch [11/25] - Loss: 0.3598
Epoch [12/25] - Loss: 0.3587
Epoch [13/25] - Loss: 0.3578
Epoch [14/25] - Loss: 0.3568
Epoch [15/25] - Loss: 0.3562
Epoch [16/25] - Loss: 0.3546
Epoch [17/25] - Loss: 0.3541

```

```
Epoch [18/25] - Loss: 0.3529
Epoch [19/25] - Loss: 0.3525
Epoch [20/25] - Loss: 0.3517
Epoch [21/25] - Loss: 0.3513
Epoch [22/25] - Loss: 0.3510
Epoch [23/25] - Loss: 0.3503
Epoch [24/25] - Loss: 0.3499
Epoch [25/25] - Loss: 0.3500
CPU times: user 18.8 s, sys: 233 ms, total: 19.1 s
Wall time: 19 s
```

1.4.4 Step 3: The Evaluation

(part 3.1) Evaluating your model with the entire test dataset:

- Load your trained model `model-ann-11-5-4-1-pt` from the file, and have it predict the entire test set you have at hand: (`X_test_scaled`). Luckily, for each of the test sample in the set, you also have ground true `Exited` value in the `y_test`.
- Please report/print your model's predictive performance on the test set in terms of **accuracy**, **precision**, **recall**, and **F1 scores**.

```
[13]: from sklearn.metrics import classification_report

# load the model
model = torch.load("saved_models/model-ann-11-5-4-1-pt.ann", weights_only=False)
model.eval() # Set the model to evaluation mode (again, just in case)

# make the predictions
y_pred = torch.tensor([])
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in test_loader:
        batch_y_pred = model(batch_x)
        y_pred = torch.cat((y_pred, batch_y_pred), dim=0)

# convert to labels (threshold = 0.5)
y_pred = (y_pred >= 0.5)

# print the results
print(classification_report(y_test, y_pred, target_names=["Not Exited",
↪ "Exited"]))
```

	precision	recall	f1-score	support
Not Exited	0.86	0.94	0.90	1438
Exited	0.65	0.41	0.50	362
accuracy			0.84	1800
macro avg	0.76	0.68	0.70	1800
weighted avg	0.82	0.84	0.82	1800

(part 3.2) Evaluating your model with 1 test sample with known Exited value

- Here is a single test sample for which we know the ground true Exited value:

CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Exited
55443342	Reynolds	709	Germany	Male	30	9	115479.48	2	1	1	134732.99	0

- Load your trained model `model-ann-11-5-4-1-pt` from the file, and have it predict the test sample above. Please don't forget to preprocess this test samples so that it is compliant with the input and model requirements.
- Please report whether it predicts a 0 or 1 for the Exited target, and also comment whether your model makes a mistake or predicts correctly.

```
[14]: # load the model
model = torch.load("saved_models/model-ann-11-5-4-1-pt.ann", weights_only=False)
model.eval() # set the model to evaluation mode (again again, just in case)

# create an empty dataframe with the same columns as the original dataset
df_3_2 = dataset_dropped.iloc[:0].copy()

# add the single new row to the dataframe
new_row = {
    "CreditScore": 709,
    "Geography": "Germany",
    "Gender": "Male",
    "Age": 30,
    "Tenure": 9,
    "Balance": 115479.48,
    "NumOfProducts": 2,
    "HasCrCard": 1,
    "IsActiveMember": 1,
    "EstimatedSalary": 134732.99
}
df_3_2.loc[0] = new_row

# a single row of y_true
y_true_3_2 = [0]

# preprocess the new row using the same preprocessor
X_3_2 = preprocessor.transform(df_3_2)

# make a dataloader
dataset_3_2 = BankChurnDataset(X_3_2, y_true_3_2)
loader_3_2 = DataLoader(dataset_3_2, batch_size=1, shuffle=False)

# make the prediction
```

```

y_pred_3_2 = None
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in loader_3_2:
        y_pred_3_2 = model(batch_x).item() # get the single prediction as a
        ↪ float

# print the result
predicted_label_3_2 = "Exited" if y_pred_3_2 >= 0.5 else "Not Exited"
predicted_probability_3_2 = y_pred_3_2 * 100
print(f"Predicted label: {predicted_label_3_2} ({predicted_probability_3_2:.
    ↪ 2f})% chance of exiting)")

```

Predicted label: Not Exited (1.12% chance of exiting)

(part 3.3) Evaluating your model with 1 test sample without known Exited value

- Here is a single test sample for which we **do not know** the ground true Exited value:

CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
5544332	Nguyen	603	France	Female	76	20	123456.78	5	1	1	55000.00

- Load your trained model `model-ann-11-5-4-1-pt` from the file, and have it predict the test sample above. Please don't forget to preprocess this test samples so that it is compliant with the input and model requirements.
- Please report whether it predicts a 0 or 1 for the Exited target. Can you comment on this data sample whether your model captured the pattern in the population?

```

[15]: # load the model
model = torch.load("saved_models/model-ann-11-5-4-1-pt.ann", weights_only=False)
model.eval() # set the model to evaluation mode (again again, just in case)

# create an empty dataframe with the same columns as the original dataset
df_3_3 = dataset_dropped.iloc[:0].copy()

# add the single new row to the dataframe
new_row = {
    "CreditScore": 603,
    "Geography": "France",
    "Gender": "Female",
    "Age": 76,
    "Tenure": 20,
    "Balance": 123456.78,
    "NumOfProducts": 5,
    "HasCrCard": 1,
    "IsActiveMember": 1,
    "EstimatedSalary": 55000.00
}
df_3_3.loc[0] = new_row

```

```

# a single row of y_true
y_true_3_3 = [0]

# preprocess the new row using the same preprocessor
X_3_3 = preprocessor.transform(df_3_3)

# make a dataloader
dataset_3_3 = BankChurnDataset(X_3_3, y_true_3_3)
loader_3_3 = DataLoader(dataset_3_3, batch_size=1, shuffle=False)

# make the prediction
y_pred_3_3 = None
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in loader_3_3:
        y_pred_3_3 = model(batch_x).item() # get the single prediction as a
        ↪ float

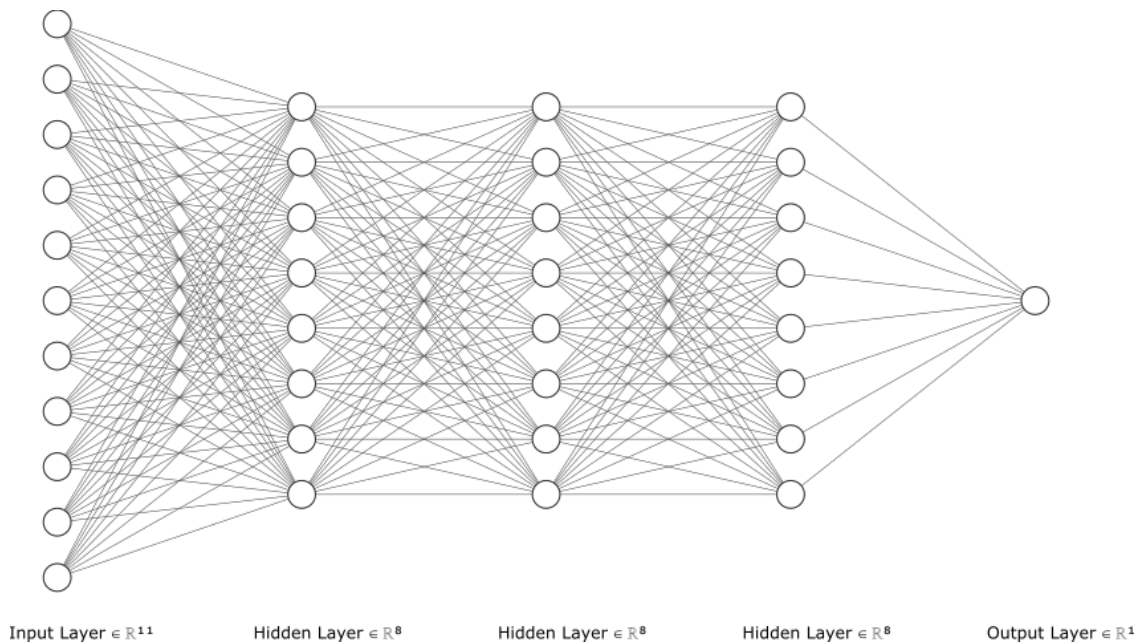
# print the result
predicted_label_3_3 = "Exited" if y_pred_3_3 >= 0.5 else "Not Exited"
predicted_probability_3_3 = y_pred_3_3 * 100
print(f"Predicted label: {predicted_label_3_3} ({predicted_probability_3_3:.
    ↪ 2f}% chance of exiting)")

```

Predicted label: Exited (75.52% chance of exiting)

1.5 Task 4: (10 points)

- Repeat all the steps in Task 3 with the following new architecture of the neural network:



- Input layer will still have 11 units as the dimension of training set (i.e, number of columns = 11).
- Hidden-layer-1: 8 neurons, with relu activation
- Hidden-layer-2: 8 neurons, with relu activation,
- Hidden-layer-3: 8 neurons, with relu activation,
- Output-layer: 1 neuron with sigmoid.
- You may re-use the Dataset/Dataloader classes you defined in Task 3.

```
[22]: %%time

class BankChurnANN_4(nn.Module):

    def __init__(self):

        super().__init__()

        # define the layers
        self.hidden1 = nn.Linear(11, 8)
        self.hidden2 = nn.Linear(8, 8)
        self.hidden3 = nn.Linear(8, 8)
        self.output = nn.Linear(8, 1)

        # define the activation functions
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # pass the data through layers and activations
        x = self.relu(self.hidden1(x))
        x = self.relu(self.hidden2(x))
        x = self.relu(self.hidden3(x))

        # sigmoid to return a probability between 0 and 1
        x = self.sigmoid(self.output(x))

        # return the output
        return x.squeeze(-1) # remove any extra dimensions for compatibility
        ↪with BCELoss

# make an instance of the model
model_4 = BankChurnANN_4()
# define the loss function and optimizer
loss_function_4 = nn.BCELoss()
```

```
optimizer_4 = optim.Adam(model_4.parameters(), lr=0.001)

# train the model and save it
train_model(model_4, train_loader, loss_function_4, optimizer_4, n_epochs,
            ↪ "saved_models/model-ann-11-8-8-8-1-pt.ann")
```

```
Epoch [1/25] - Loss: 0.5453
Epoch [2/25] - Loss: 0.4157
Epoch [3/25] - Loss: 0.3967
Epoch [4/25] - Loss: 0.3875
Epoch [5/25] - Loss: 0.3802
Epoch [6/25] - Loss: 0.3694
Epoch [7/25] - Loss: 0.3605
Epoch [8/25] - Loss: 0.3524
Epoch [9/25] - Loss: 0.3478
Epoch [10/25] - Loss: 0.3461
Epoch [11/25] - Loss: 0.3443
Epoch [12/25] - Loss: 0.3423
Epoch [13/25] - Loss: 0.3418
Epoch [14/25] - Loss: 0.3416
Epoch [15/25] - Loss: 0.3402
Epoch [16/25] - Loss: 0.3390
Epoch [17/25] - Loss: 0.3382
Epoch [18/25] - Loss: 0.3385
Epoch [19/25] - Loss: 0.3373
Epoch [20/25] - Loss: 0.3358
Epoch [21/25] - Loss: 0.3358
Epoch [22/25] - Loss: 0.3358
Epoch [23/25] - Loss: 0.3348
Epoch [24/25] - Loss: 0.3359
Epoch [25/25] - Loss: 0.3341
CPU times: user 21.6 s, sys: 24.8 ms, total: 21.6 s
Wall time: 21.6 s
```

```
[23]: # do the same for the new sample #1

# make a dataloader
loader_4_2 = loader_3_2 # reuse the dataloader from the previous example

# put the model in evaluation mode
model_4.eval()

# make the prediction
y_pred_4_2 = None
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in loader_4_2:
```

```

        y_pred_4_2 = model_4(batch_x).item() # get the single prediction as a
↪float

# print the result
predicted_label_4_2 = "Exited" if y_pred_4_2 >= 0.5 else "Not Exited"
predicted_probability_4_2 = y_pred_4_2 * 100
print(f"New sample #1 predicted label: {predicted_label_4_2}
↪({predicted_probability_4_2:.2f}% chance of exiting)")

```

New sample #1 predicted label: Not Exited (0.98% chance of exiting)

```

[24]: # do the same for new sample #2

# make a dataloader
loader_4_3 = loader_3_3 # reuse the dataloader from the previous example

# put the model in evaluation mode
model_4.eval()

# make the prediction
y_pred_4_3 = None
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in loader_4_3:
        batch_x = batch_x
        y_pred_4_3 = model_4(batch_x).item() # get the single prediction as a
↪float

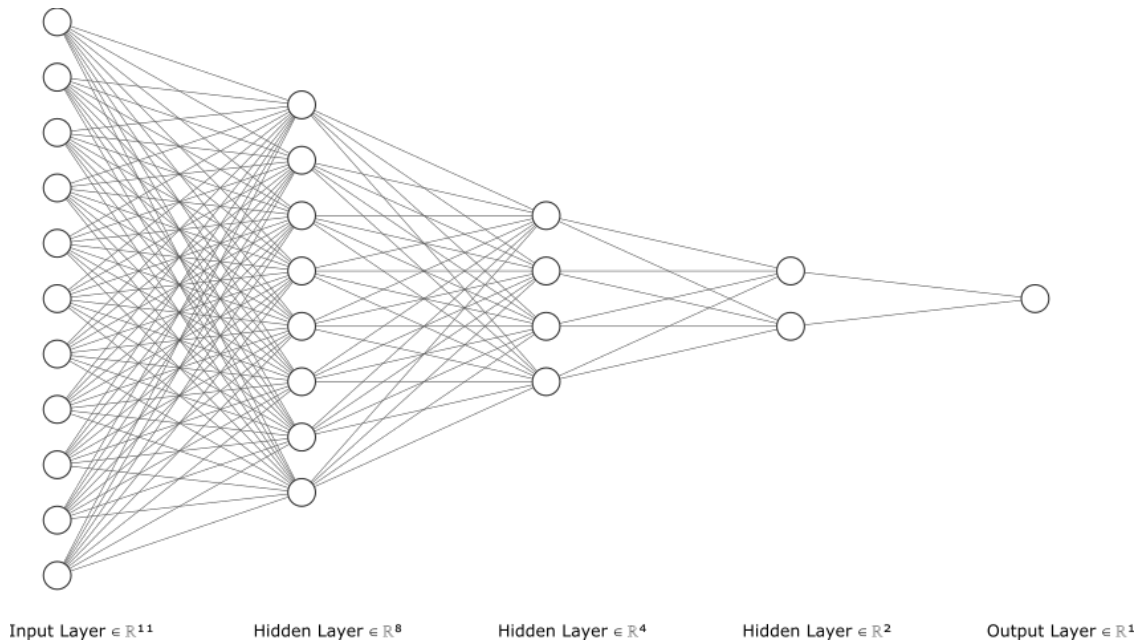
# print the result
predicted_label_4_3 = "Exited" if y_pred_4_3 >= 0.5 else "Not Exited"
predicted_probability_4_3 = y_pred_4_3 * 100
print(f"New sample #2 predicted label: {predicted_label_4_3}
↪({predicted_probability_4_3:.2f}% chance of exiting)")

```

New sample #2 predicted label: Exited (99.66% chance of exiting)

1.6 Task 5: (10 points)

- Again, repeat Task 3 with the following new architecture of the neural network:



- Input layer will still have 11 units as the dimension of training set (i.e, number of columns = 11).
- Hidden-layer-1: 8 neurons, with relu activation
- Hidden-layer-2: 4 neurons, with relu activation,
- Hidden-layer-3: 2 neurons, with relu activation,
- Output-layer: 1 neuron with sigmoid.

```
[25]: %%time

class BankChurnANN_5(nn.Module):

    def __init__(self):

        super().__init__()

        # define the layers
        self.hidden1 = nn.Linear(11, 8)
        self.hidden2 = nn.Linear(8, 4)
        self.hidden3 = nn.Linear(4, 2)
        self.output = nn.Linear(2, 1)

        # define the activation functions
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # pass the data through layers and activations
        x = self.relu(self.hidden1(x))
```

```

        x = self.relu(self.hidden2(x))
        x = self.relu(self.hidden3(x))

        # sigmoid to return a probability between 0 and 1
        x = self.sigmoid(self.output(x))

        # return the output
        return x.squeeze(-1) # remove any extra dimensions for compatibility
        ↪with BCELoss

# make an instance of the model
model_5 = BankChurnANN_5()
# define the loss function and optimizer
loss_function_5 = nn.BCELoss()
optimizer_5 = optim.Adam(model_5.parameters(), lr=0.001)

# train the model and save it
train_model(model_5, train_loader, loss_function_5, optimizer_5, n_epochs,
        ↪"saved_models/model-ann-11-8-4-2-1-pt.ann")

```

```

Epoch [1/25] - Loss: 0.5307
Epoch [2/25] - Loss: 0.4349
Epoch [3/25] - Loss: 0.4191
Epoch [4/25] - Loss: 0.4081
Epoch [5/25] - Loss: 0.3980
Epoch [6/25] - Loss: 0.3881
Epoch [7/25] - Loss: 0.3810
Epoch [8/25] - Loss: 0.3753
Epoch [9/25] - Loss: 0.3690
Epoch [10/25] - Loss: 0.3625
Epoch [11/25] - Loss: 0.3558
Epoch [12/25] - Loss: 0.3513
Epoch [13/25] - Loss: 0.3486
Epoch [14/25] - Loss: 0.3456
Epoch [15/25] - Loss: 0.3439
Epoch [16/25] - Loss: 0.3432
Epoch [17/25] - Loss: 0.3420
Epoch [18/25] - Loss: 0.3410
Epoch [19/25] - Loss: 0.3400
Epoch [20/25] - Loss: 0.3396
Epoch [21/25] - Loss: 0.3384
Epoch [22/25] - Loss: 0.3377
Epoch [23/25] - Loss: 0.3368
Epoch [24/25] - Loss: 0.3356
Epoch [25/25] - Loss: 0.3354
CPU times: user 19.1 s, sys: 34 ms, total: 19.1 s
Wall time: 19.1 s

```

```
[26]: # do the same for the new sample #1

# make a dataloader
loader_5_2 = loader_3_2 # reuse the dataloader from the previous example

# put the model in evaluation mode
model_5.eval()

# make the prediction
y_pred_5_2 = None
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in loader_5_2:
        y_pred_5_2 = model_5(batch_x).item() # get the single prediction as a
        ↪ float

# print the result
predicted_label_5_2 = "Exited" if y_pred_5_2 >= 0.5 else "Not Exited"
predicted_probability_5_2 = y_pred_5_2 * 100
print(f"New sample #1 predicted label: {predicted_label_5_2}
      ↪ ({predicted_probability_5_2:.2f}% chance of exiting)")
```

New sample #1 predicted label: Not Exited (1.06% chance of exiting)

```
[27]: # do the same for new sample #2

# make a dataloader
loader_5_3 = loader_3_3 # reuse the dataloader from the previous example

# put the model in evaluation mode
model_5.eval()

# make the prediction
y_pred_5_3 = None
with torch.no_grad(): # disable gradient calculation
    for batch_x, _ in loader_5_3:
        y_pred_5_3 = model_5(batch_x).item() # get the single prediction as a
        ↪ float

# print the result
predicted_label_5_3 = "Exited" if y_pred_5_3 >= 0.5 else "Not Exited"
predicted_probability_5_3 = y_pred_5_3 * 100
print(f"New sample #2 predicted label: {predicted_label_5_3}
      ↪ ({predicted_probability_5_3:.2f}% chance of exiting)")
```

New sample #2 predicted label: Exited (99.34% chance of exiting)

2 That’s all folks! Thanks for your effort.

Now, do the following to earn credit –

0. Setting up:
 - Make sure you actually experiment with this assignment. I would encourage (again) you to go through the possible compute resource you can use (e.g., Kaggle, Google Colab, etc.).
 - It’s always better to work in python virtual environment.
1. Please make sure to execute each cell in this jupyter notebook, and hit the ‘Save’ button, or go “File > Save and Checkpoint” menu option to save the notebook.
2. Submit this notebook “2026-Fall-DL-hw2.ipynb” into Canvas “hw-2” entry.
3. Done!